고급통계분석 hw1

20240769 장선영

2026-10-13

Problem 1

(a)

```
set.seed(1)

n = 200
x = seq(0, 1, length.out = n)
y = sin(2*pi*x) + rnorm(n, sd = 0.15)

df = data.frame(x, y)

head(df)
```

```
##             x           y
## 1 0.000000000 -0.09396807
## 2 0.005025126  0.05911505
## 3 0.010050251 -0.06223866
## 4 0.015075377  0.33387193
## 5 0.020100503  0.17538587
## 6 0.025125628  0.03414379
```

```
write.csv(df, "data/simulation.csv", row.names = FALSE)
```

(b)

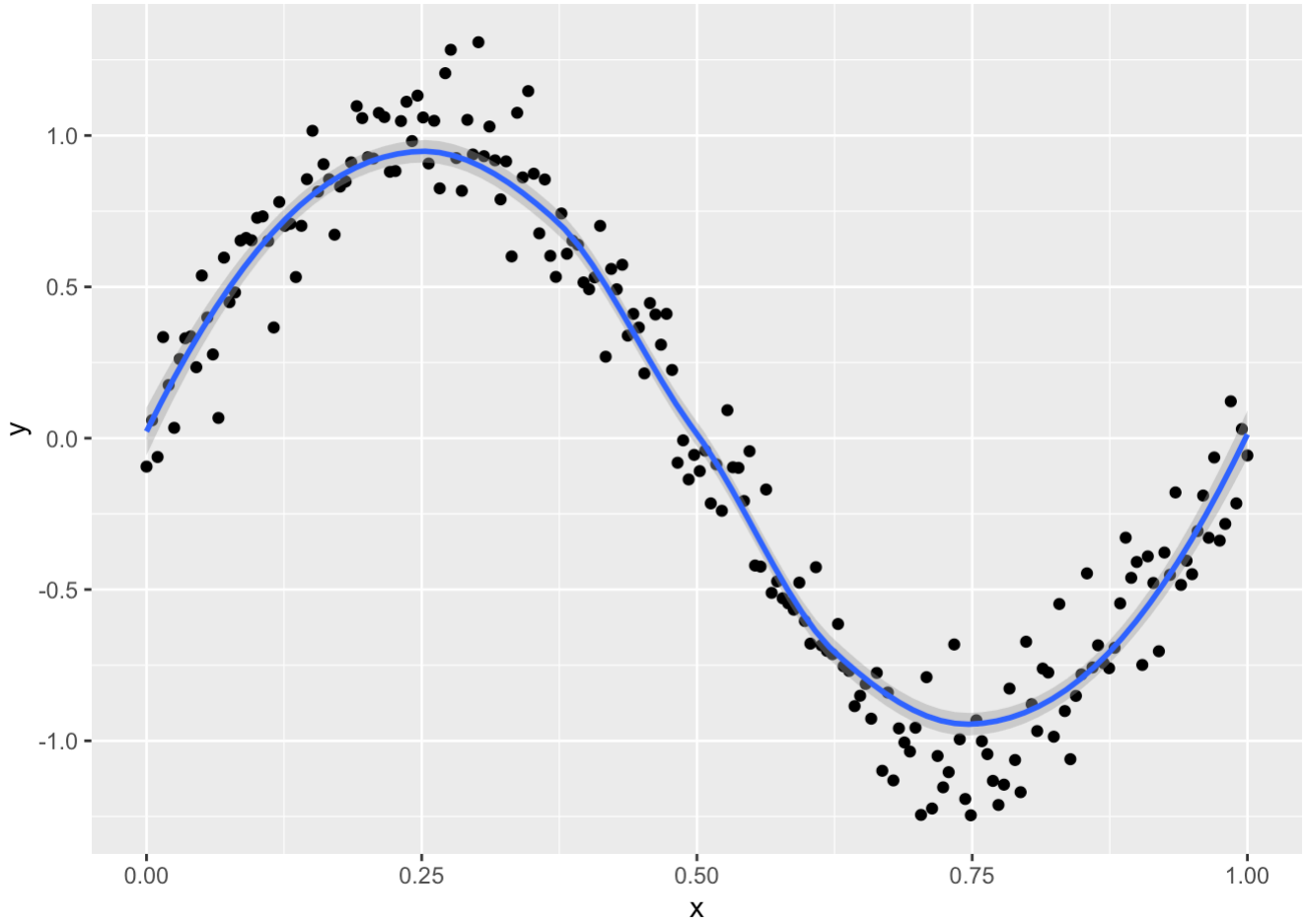
```
library(ggplot2)

dat = read.csv("data/simulation.csv")

p = ggplot(dat, aes(x = x, y = y)) +
  geom_point() +
  geom_smooth(method = "loess")

print(p)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



(c)

```
ggsave(
  filename = "plots/simulation.png",
  plot = p,
  width = 7,
  height = 5
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

(d)

https://github.com/sunnyoung05/Assignment1 (https://github.com/sunnyoung05/Assignment1)

Problem 2

(a)

```
bubble_sort = function(x, decreasing=FALSE) {
  n = length(x)
  if (n <= 1) {
    return(x)
  }
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (decreasing == FALSE) {
        if (x[j] > x[j+1]) {
          temp = x[j]
          x[j] = x[j+1]
          x[j+1] = temp
        }
      } else {
        if (x[j] < x[j+1]) {
          temp = x[j]
          x[j] = x[j+1]
          x[j+1] = temp
        }
      }
    }
  }
  return(x)
}

set.seed(1)
x = runif(10)

bubble_sort(x, decreasing=FALSE)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
bubble_sort(x, decreasing=TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

(b)

```
quick_sort = function(x, decreasing=FALSE) {
  if (length(x) <= 1) {
    return(x)
  }
  pivot = x[1]
  if (decreasing == FALSE) {
    left = x[x < pivot]
    middle = x[x == pivot]
    right = x[x > pivot]
  } else {
    left = x[x > pivot]
    middle = x[x == pivot]
    right = x[x < pivot]
  }
  result = c(quick_sort(left, decreasing), middle, quick_sort(right, decreasing))
  return(result)
}

set.seed(1)
x = runif(10)

quick_sort(x, decreasing=FALSE)
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
quick_sort(x, decreasing=TRUE)
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

Problem 3

(a)

```
numerical_derivative = function(f, x, h=1e-6, method="central") {
  if (method == "forward") {
    result = (f(x+h)-f(x))/h
  } else if (method == "backward") {
    result = (f(x)-f(x-h))/h
  } else if (method == "central") {
    result = (f(x+h)-f(x-h))/(2*h)
  } else {
    stop("Invalid method")
  }
  return(result)
}

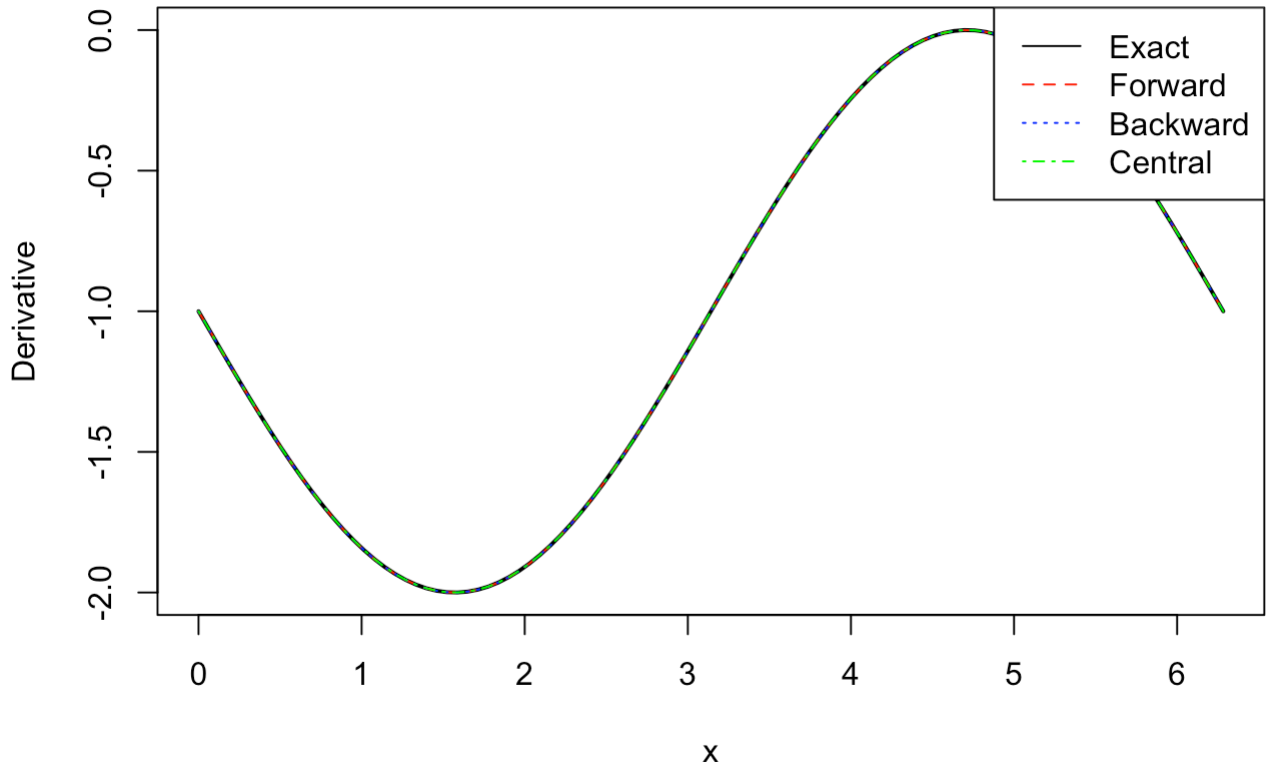
f = function(x) cos(x)-x
fprime = function(x) -sin(x)-1

x = seq(0, 2*pi, length.out=100)

forward = numerical_derivative(f, x, method="forward")
backward = numerical_derivative(f, x, method="backward")
central = numerical_derivative(f, x, method="central")
exact = fprime(x)

plot(x, exact, type="l", col="black", lwd=2,
     xlab="x", ylab="Derivative")
lines(x, forward, col="red", lty=2)
lines(x, backward, col="blue", lty=3)
lines(x, central, col="green", lty=4)

legend("topright",
       legend=c("Exact", "Forward", "Backward", "Central"),
       col=c("black", "red", "blue", "green"),
       lty=c(1, 2, 3, 4))
```



(b)

```
newton_raphson = function(f, x0, fprime=NULL, maxiter=100,
                          h=1e-6, epsilon=1e-10) {
  x = x0
  for (i in 1:maxiter) {
    if (is.null(fprime)) {
      df = numerical_derivative(f, x, h=h, method="central")
    } else {
      df = fprime(x)
    }
    if (abs(df) < 1e-14) {
      stop("Derivative is too small")
    }
    x_new = x-f(x)/df
    if (abs(x_new-x) < epsilon) {
      return(x_new)
    }
    x = x_new
  }
  return(x)
}
```

(c)

```
f = function(x) cos(x)-x
fprime = function(x) -sin(x)-1

result1 = newton_raphson(f, x0=0.5)
result2 = newton_raphson(f, x0=0.5, fprime=fprime)

result1
```

```
## [1] 0.7390851
```

```
result2
```

```
## [1] 0.7390851
```

Problem 4

(a)

```
left_rectangle = function(f, a, b, n) {
  h = (b-a)/n
  result = 0
  for (i in 0:(n-1)) {
    result = result+f(a+i*h)
  }
  return(h*result)
}
```

(b)

```
trapezoid = function(f, a, b, n) {
  h = (b-a)/n
  result = (f(a)+f(b))/2
  for (i in 1:(n-1)) {
    result = result+f(a+i*h)
  }
  return(h*result)
}
```

(c)

```
simpson = function(f, a, b, n) {
  if (n%%2 != 0) {
    stop("n must be even")
  }
  h = (b-a)/n
  result = f(a)+f(b)
  for (i in 1:(n-1)) {
    if (i%%2 == 1) {
      result = result+4*f(a+i*h)
    } else {
      result = result+2*f(a+i*h)
    }
  }
  return(h*result/3)
}
```

(d)

```
f = function(x) sin(x)
n = 100

left_rectangle(f, 0, pi, n)
```

```
## [1] 1.999836
```

```
trapezoid(f, 0, pi, n)
```

```
## [1] 1.999836
```

```
simpson(f, 0, pi, n)
```

```
## [1] 2
```

(e)

```
n_values = c(10, 30, 60, 100, 150, 200)

error_left = numeric(length(n_values))
error_trap = numeric(length(n_values))
error_simp = numeric(length(n_values))

for (i in 1:length(n_values)) {
  n = n_values[i]
  error_left[i] = abs(left_rectangle(f, 0, pi, n)-2)
  error_trap[i] = abs(trapezoid(f, 0, pi, n)-2)
  error_simp[i] = abs(simpson(f, 0, pi, n)-2)
}

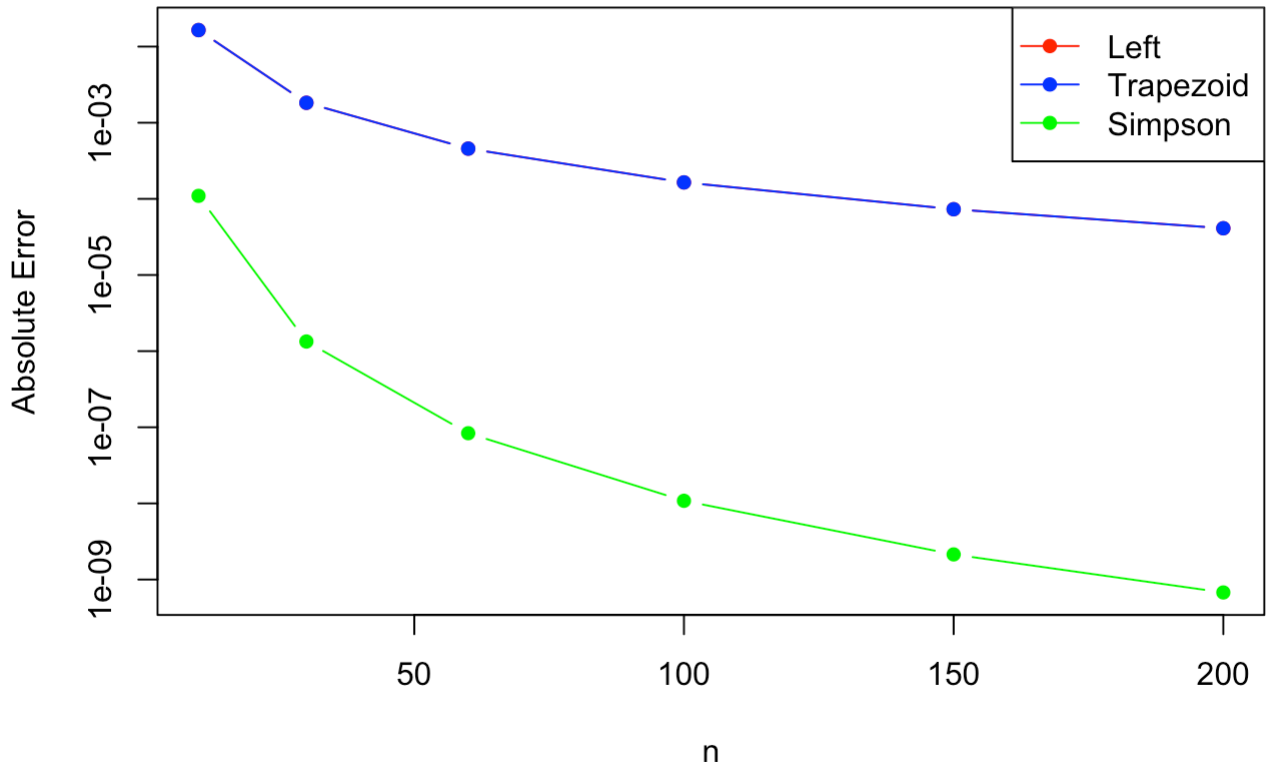
result = data.frame(
  n=n_values,
  Left=error_left,
  Trapezoid=error_trap,
  Simpson=error_simp
)

print(result)
```

```
##     n         Left    Trapezoid      Simpson
## 1  10 1.647646e-02 1.647646e-02 1.095173e-04
## 2  30 1.828039e-03 1.828039e-03 1.337948e-06
## 3  60 4.569470e-04 4.569470e-04 8.353986e-08
## 4 100 1.644961e-04 1.644961e-04 1.082450e-08
## 5 150 7.310872e-05 7.310872e-05 2.138034e-09
## 6 200 4.112352e-05 4.112352e-05 6.764731e-10
```

```
matplot(n_values, cbind(error_left, error_trap, error_simp),
        type="b", pch=16, lty=1,
        col=c("red", "blue", "green"),
        xlab="n", ylab="Absolute Error",
        log="y")

legend("topright",
       legend=c("Left", "Trapezoid", "Simpson"),
       col=c("red", "blue", "green"),
       lty=1, pch=16)
```



Problem 5

(a)

```
A = matrix(c(4,2,2,2,5,1,2,1,3), 3)

L = t(chol(A))

L
```

```
##      [,1] [,2]     [,3]
## [1,]    2    0 0.000000
## [2,]    1    2 0.000000
## [3,]    1    0 1.414214
```

```
L%*%t(L)
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

(b)

```
forward = function(L, b) {
  n = length(b)
  z = numeric(n)
  for (i in 1:n) {
    s = 0
    if (i > 1) {
      for (j in 1:(i-1)) {
        s = s+L[i,j]*z[j]
      }
    }
    z[i] = (b[i]-s)/L[i,i]
  }
  return(z)
}
```

(c)

```
backward = function(U, z) {
  n = length(z)
  x = numeric(n)
  for (i in n:1) {
    s = 0
    if (i < n) {
      for (j in (i+1):n) {
        s = s+U[i,j]*x[j]
      }
    }
    x[i] = (z[i]-s)/U[i,i]
  }
  return(x)
}
```

(d)

```
b = c(1,-2,3)

z = forward(L,b)
x = backward(t(L),z)

x
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
solve(A,b)
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
all.equal(x,as.vector(solve(A,b)))
```

```
## [1] TRUE
```

Problem 6

(a)

```
gaussian_kernel = function(x, y, rho=1) {
  return(exp(-rho*sum((x-y)^2)))
}
```

(b)

```
krr = function(X, y, lambda=0.0001, rho=1) {
  X = as.matrix(X)
  n = nrow(X)
  K = matrix(0,n,n)
  for (i in 1:n) {
    for (j in 1:n) {
      K[i,j] = gaussian_kernel(X[i,],X[j,],rho)
    }
  }
  alpha = solve(K+lambda*diag(n),y)
  result = list(X=X,y=y,alpha=alpha,rho=rho,lambda=lambda)
  class(result) = "krr"
  return(result)
}
```

(c)

```
predict.krr = function(object, newdata, ...) {
  X = object$X
  newdata = as.matrix(newdata)
  n = nrow(X)
  m = nrow(newdata)
  K_new = matrix(0,m,n)
  for (i in 1:m) {
    for (j in 1:n) {
      K_new[i,j] = gaussian_kernel(newdata[i,],X[j,],object$rho)
    }
  }
  return(as.vector(K_new%*%object$alpha))
}
```

(d)

```
plot.krr = function(x, ...) {
  X = x$X[,1]
  y = x$y
  x_grid = seq(min(X),max(X),length.out=200)
  y_pred = predict(x,matrix(x_grid,ncol=1))
  plot(X,y,pch=16,xlab="x",ylab="y")
  lines(x_grid,y_pred,col="red",lwd=2)
}
```

(e)

```
set.seed(1)
n = 150
X = matrix(runif(n,-1,1),ncol=1)
ftrue = function(x) sin(2*pi*x)+0.5*cos(4*pi*x)
y = ftrue(X[,1])+rnorm(n,sd=0.1)

model = krr(X,y)

x_new = matrix(seq(-1,1,length.out=200),ncol=1)
y_pred = predict(model,x_new)

plot(model)
```

